Consultas Aninhadas em SQL

Em alguns casos, precisamos realizar uma consulta que é comparada com o resultado de outra consulta. Aqui teremos o uso de uma **consulta dentro de outra consulta** ou **consulta aninhada**. A consulta que é realizada dentro de outra é chamada subconsulta.

Uma **subconsulta, consulta interna ou seleção interna** é uma consulta que está aninhada dentro de uma instrução SELECT, INSERT, UPDATE ou DELETE ou em outra subconsulta. Uma subconsulta pode ser usada em qualquer lugar em que é permitida uma expressão.

As subconsultas podem ser comparadas com a consulta externa com o uso de operadores IN (ou NOT IN), ANY, ALL e EXISTS (ou NOT EXISTS), além dos operadores básicos =, <, <=, >, >=, <>.

OBS: Em SQLite não existe ANY e ALL

EXEMPLIFICANDO!!!

Dada a tabela Produtos a seguir:

ProdutoID	Nome_do_Produto	FornecedorID	CategoriaID	Unidade	Preco
1	Leite	1	1	Litros	3
2	Banana	1	1	Kilogramas	5
3	Melancia	1	2	Unidade	6
4	Pão	2	2	Pacote	4
5	Suco	2	2	Litros	8

E a tabela Produtos Líquidos a seguir:

ProdutoID	Nome_do_Produto
1	Leite
5	Suco

Vamos supor que você deseje consultar o preço médio de seus produtos líquidos. Para isso, você pode fazer uma consulta aninhada:

```
SELECT AVG(Preco)
FROM Produtos
WHERE ProdutoID IN (SELECT ProdutoID FROM ProdutosLiquidos);
```

O retorno dessa consulta será 5,5, pois a subconsulta retornará os ids 1 e 5 que serão utilizados pela consulta externa para fazer a média dos preços.

EXISTS (e NOT EXISTS)

A cláusula **EXISTS** faz uma **verificação se existe algum resultado para a subconsulta informada**. **Caso haja**, o **resultado da consulta principal é exibido**. É muito comum sua utilização quando se deseja trazer resultados onde um valor específico existe dentro de outra tabela. A sintaxe básica é:

> **SELECT** colunas **FROM** tabela **WHERE EXISTS** (**SELECT** colunas **FROM** tabela WHERE condicão);

Da mesma forma, também há a cláusula **NOT EXISTS**, somente **retorna o resultado da consulta principal, se não houver nenhum resultado para a subconsulta**.

EXEMPLIFICANDO!!!

Considere as tabelas a seguir:

PRODUTO

id	nome	preco	id_categoria
1	Bola	35.00	1
2	Patinete	120.00	1
3	Carrinho	15.00	1
4	Skate	296.00	1
5	Notebook	3500.00	2
6	Monitor LG 19	450.00	2
7	O Diário de Anne Frank	45.00	3
8	O dia do Curinga	65.00	3
9	O mundo de Sofia	48.00	3
10	Através do Espelho	38.00	3

VENDA_PRODUTO

id	id_produto	valor	data
1	1	35.00	15/05/2018
2	1	35.00	15/06/2018
3	1	35.00	15/07/2018
4	2	120.00	15/07/2018
5	2	120.00	14/07/2018
6	3	15.00	15/07/2018
7	7	45.00	15/07/2018
8	8	65.00	15/07/2018
9	8	65.00	16/07/2018
10	9	48.00	16/07/2018
11	5	3500.00	16/07/2018
12	5	3500.00	16/07/2018
13	6	450.00	16/07/2018

Suponha que seja necessário trazer em uma consulta na tabela de produtos, todos aqueles registros que tiveram alguma venda. Para isso podemos utilizar o EXISTS, além de testar se a condição é verdadeira, traz como retorno os dados da consulta. A sintaxe a seguir poderá ser utilizada:

SELECT p.id, p.nome **FROM** produto p **WHERE EXISTS** (**SELECT** v.id_produto **FROM** venda_produto v WHERE v.id_produto = p.id);

Vamos analisar o comando por partes:

SELECT p.id, p.nome **FROM** produto p

-- seleção do id e nome da tabela produto

WHERE EXISTS

-- a consulta só será efetuada para o registro que cumprir a condição da subconsulta

(**SELECT** v.id_produto **FROM** venda_produto v WHERE v.id_produto = p.id);

-- isto é, somente se o id do produto constar na tabela venda_produto.

O resultado desse comando será:

id	nome
1	Bola
2	Patinete
3	Carrinho
5	Notebook
6	Monitor LG 19
7	O Diário de Anne Frank
8	O dia do Curinga
9	O mundo de Sofia

Vamos entender o que aconteceu no comando passo a passo:

1. Isolando o primeiro produto e comparando-o com todos os resultados da subconsulta com base na condição WHERE, ou seja, verificar se o valor do seu id é igual ao valor de algum id da tabela venda_produto (v.id_produto = p.id).

PRODUTO

id	nome	preco	id_categoria
1	Bola	35.00	1
2	Patinete	120.00	1
3	Carrinho	15.00	1
4	Skate	296.00	1
5	Notebook	3500.00	2
6	Monitor LG 19	450.00	2
7	O Diário de Anne Frank	45.00	3
8	O dia do Curinga	65.00	3
9	O mundo de Sofia	48.00	3
10	Através do Espelho	38.00	3

VENDA_PRODUTO

id	id_produto	valor	data	
1	1	35.00	15/05/2018	1=1? SIM
2	1	35.00	15/06/2018	
3	1	35.00	15/07/2018	
4	2	120.00	15/07/2018	
5	2	120.00	14/07/2018	
6	3	15.00	15/07/2018	
7	7	45.00	15/07/2018	
8	8	65.00	15/07/2018	
9	8	65.00	16/07/2018	
10	9	48.00	16/07/2018	
11	5	3500.00	16/07/2018	
12	5	3500.00	16/07/2018	
13	6	450.00	16/07/2018	

Como existe um valor correspondente, então o EXISTS retorna TRUE, e o SELECT será executado para essa primeira linha de id = 1.

2. Perceba que com 2 e 3 acontecerá a mesma coisa. Ilustrando com o 2:

PRODUTO

id	nome	preco	id_categoria
1	Bola	35.00	1
2	Patinete	120.00	1
3	Carrinho	15.00	1
4	Skate	296.00	1

VENDA_PRODUTO

id	id_produto	valor	data	
1	1	35.00	15/05/2018	2=1? NÃO
2	1	35.00	15/06/2018	2=1? NÃO
3	1	35.00	15/07/2018	2=1? NÃO
4	2	120.00	15/07/2018	2=2? SIM

O fato é que mesmo que alguns valores não cumpram a condição, basta que um cumpra.

3. Agora vamos analisar o que acontecerá na linha do produto 4.

PRODUTO

id	nome	preco	id_categoria
1	Bola	35.00	1
2	Patinete	120.00	1
3	Carrinho	15.00	1
4	Skate	296.00	1
5	Notebook	3500.00	2
6	Monitor LG 19	450.00	2
7	O Diário de Anne Frank	45.00	3
8	O dia do Curinga	65.00	3
9	O mundo de Sofia	48.00	3
10	Através do Espelho	38.00	3

VENDA_PRODUTO

id	id_produto	valor	data	
1	1	35.00	15/05/2018	4=1? NÃO
2	1	35.00	15/06/2018	4=1? NÃO
3	1	35.00	15/07/2018	4=1? NÃO
4	2	120.00	15/07/2018	4=2? NÃO
5	2	120.00	14/07/2018	4=2? NÃO
6	3	15.00	15/07/2018	4=3? NÃO
7	7	45.00	15/07/2018	4=7? NÃO
8	8	65.00	15/07/2018	4=8? NÃO
9	8	65.00	16/07/2018	4=8? NÃO
10	9	48.00	16/07/2018	4=9? NÃO
11	5	3500.00	16/07/2018	4=5? NÃO
12	5	3500.00	16/07/2018	4=5? NÃO
13	6	450.00	16/07/2018	4=6? NÃO

Nessa situação, não há nenhum valor retornado na subconsulta, pois nenhum valor cumpre a condição de igualdade dos ids. Assim, o EXISTS retornará FALSE e o produto de id=4 não será retornado na consulta externa.

4. O processo seguirá para todos os registros de produto, mas nós já podemos perceber que os valores que serão retornados são aqueles de produtos cujos id estão também na tabela venda_produto. Logo, 1, 2, 3, 5, 6, 7, 8 e 9. Somente 4 e 10 não serão retornados.

ATENÇÃO!!!

A cláusula EXISTS retorna TRUE (verdadeiro) se existir algum registro que cumpre a subconsulta. A consulta externa, por sua vez, retorna o registro. Caso contrário, o EXISTS retorna FALSE (falso) e a consulta externa não irá exibir o registro.

Do mesmo modo, a cláusula NOT EXISTS retorna TRUE (verdadeiro) se não existir nenhum registro que cumpra a subconsulta. A consulta externa, por sua vez, retorna o registro. Caso contrário, o NOT EXISTS retorna FALSE (falso) e a consulta externa não irá exibir o registro.

Esquematizando:

Consulta aninhada

- Uma subconsulta, consulta interna ou seleção interna é uma consulta que está aninhada dentro de uma instrução SELECT, INSERT, UPDATE ou DELETE ou em outra subconsulta.

- As subconsultas podem ser comparadas com a consulta externa com o uso de operadores IN (ou NOT IN), ANY, ALL ou EXISTS (ou NOT EXISTS), além dos operadores básicos =, <, <=, >, >=, <>.

Cláusulas Especiais

Não tem no SQLite →

- **ANY**: retorna TRUE se qualquer um dos valores da subconsulta atender a condição.

Não tem no SQLite →

- **ALL**: retorna TRUE se todos os valores da subconsulta atenderem a condição.
- **EXISTS**: retorna TRUE se a subconsulta retornar um ou mais registros.
- **NOT EXISTS**: retorna TRUE se a subsconsulta não retornar nenhum registro.

36- **(FGV - 2023 – TJ SE – Analista Judiciário)** Considere uma tabela relacional TAB, com colunas A e B. A coluna A constitui a chave primária de TAB. A instância de TAB contém 100 linhas, e em todas as linhas o valor da coluna B é 10. Nesse contexto, analise o comando SQL a seguir.

```
select * from TAB t
where not exists
    (select * from TAB tt
      where t.B = tt.B and t.A > tt.A)
```

Além da linha de títulos, o número de linhas produzidas pelo comando acima é:

a) 0;

b) 1;

c) 98;

d) 99;

e) 100.

36- **(FGV - 2023 – TJ SE – Analista Judiciário)** Considere uma tabela relacional TAB, com colunas A e B. A coluna A constitui a chave primária de TAB. A instância de TAB contém 100 linhas, e em todas as linhas o valor da coluna B é 10. Nesse contexto, analise o comando SQL a seguir.

```
select * from TAB t
where not exists
    (select * from TAB tt
      where t.B = tt.B and t.A > tt.A)
```

Além da linha de títulos, o número de linhas produzidas pelo comando acima é:

a) 0;

b) 1;

c) 98;

d) 99;

e) 100.

Resolução:

Primeiro de tudo, vamos montar uma tabela TAB fictícia com base nos dados da questão. Como a questão afirma que A é chave primária e a tabela possui 100 linhas, então termos 100 valores diferentes para A, pois uma chave não pode ter valores repetidos. Para fins de resolução, podemos considerar valores de 1 a 100 (poderiam ser quaisquer outros). O valor de B para todas as linhas é 10.

Logo, chegamos a seguinte tabela TAB:

A	B
1	10
2	10
...	10
n	10
...	10
100	10

Agora vamos ao comando:

```
select * from TAB t
-- selecionar os valores da tabela TAB referenciada como t.
where not exists
-- quando não existir retorno para a subconsulta a seguir
    (select * from TAB tt
      -- selecionar os valores da tabela TAB referenciada como tt. Nesse caso, teremos, em tempo de execução, duas referências para a mesma tabela TAB, ou seja, teremos duas cópias da mesma tabela durante a consulta.
      where t.B=tt.B and t.A > tt.A)
      -- e haverá uma comparação dos atributos das referências dessas tabelas.
```

36- **(FGV - 2023 – TJ SE – Analista Judiciário)** Considere uma tabela relacional TAB, com colunas A e B. A coluna A constitui a chave primária de TAB. A instância de TAB contém 100 linhas, e em todas as linhas o valor da coluna B é 10. Nesse contexto, analise o comando SQL a seguir.

```
select * from TAB t
where not exists
    (select * from TAB tt
      where t.B = tt.B and t.A > tt.A)
```

Além da linha de títulos, o número de linhas produzidas pelo comando acima é:

a) 0;

b) 1;

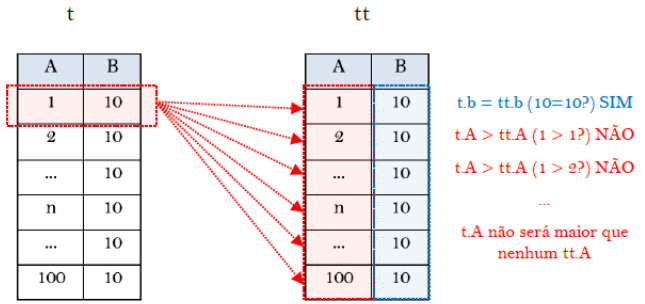
c) 98;

d) 99;

e) 100.

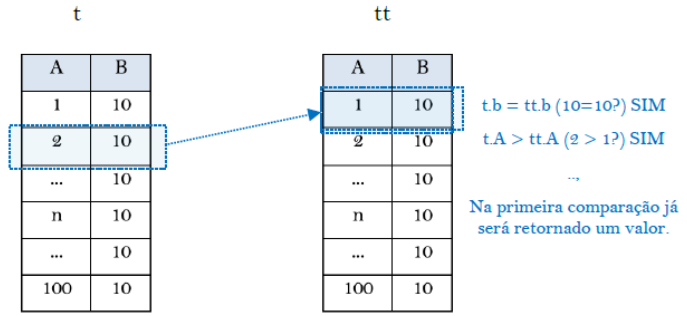
Vamos à execução:

1. Isolando o primeiro valor da referência t e comparando-o com todos os resultados da subconsulta com base na condição WHERE, ou seja, verificar se o valor de t.b é igual a tt.b e se t.A é maior que tt.A.



Como não existe nenhum registro que cumpra as condições, o NOT EXISTS retornará TRUE e, portanto, essa **primeira linha será selecionada**.

2. Vamos a segunda linha:



Nesse caso, será retornado um valor, pois existe um valor em t cujo valor de A seja maior que o valor de tt e seu B seja igual. Como existe pelo menos um registro que cumpre a condição, o NOT EXISTS retorna FALSE e, assim, o valor da **segunda linha não será retornado na consulta externa**.

3. Perceba que para todas as demais linhas, sempre haverá um retorno, pois o valor de A em t será maior do que algum valor de A da referência tt. Logo, NOT EXISTS será FALSE para todas elas e, portanto, **não será mais retornado nenhum valor**.

Embora tenhamos usado um exemplo de tabela com números de exemplo, essa conclusão pode ser obtida pela lógica. Como A é chave primária e nunca terá dois valores iguais, então teremos um menor valor que, portanto, não será maior que nenhum outro valor dessa tabela. O caso do B é mais simples, pois como sempre é 10, seu valor sempre será igual a algum valor dessa tabela (no caso a todos).

Gabarito: **Letra B.**

37- (CESPE / CEBRASPE - 2022 - TCE-SC - Auditor Fiscal de Controle Externo).

tabela1
campo
0
2
3
3
4
4
6
7

tabela2
campo
5
5
7
8
8
9

Considerando as tabela1 e tabela2 apresentadas, julgue o item que se segue, referentes a banco de dados.

Considere que o comando a seguir seja executado sem erro.

select campo from tabela2

where exists

(select campo from tabela1)

Nesse caso, o resultado será a tabela seguinte.

campo
5
5
8
8
9

Verdadeiro ou falso?

37- (CESPE / CEBRASPE - 2022 - TCE-SC - Auditor Fiscal de Controle Externo).

tabela1
campo
0
2
3
3
4
4
6
7

tabela2
campo
5
5
7
8
8
9

Considerando as tabela1 e tabela2 apresentadas, julgue o item que se segue, referentes a banco de dados.

Considere que o comando a seguir seja executado sem erro.

select campo from tabela2

where exists

(select campo from tabela1)

Nesse caso, o resultado será a tabela seguinte.

campo
5
5
8
8
9

Verdadeiro ou falso?

Resolução:

A cláusula **EXISTS** faz uma verificação se existe algum resultado para a subconsulta informada. **Caso haja**, o resultado da consulta principal é exibido.

Dado o comando:

select campo from tabela2

— selecionar o valor de campo da tabela2

where exists

— desde que esse valor exista em algum retorno da subsconsulta subsequente

(select campo from tabela1)

— selecionar o valor de campo da tabela1

O ponto desse comando é que a subconsulta apenas verifica se existe algum valor para campo na tabela1, não fazendo qualquer comparação entre os valores das tabelas. Nesse caso, a subconsulta sempre retorna TRUE, pois existem valores na tabela1. Logo, o SELECT externo será realizado para todos os valores da tabela2.

Para entender melhor, vamos isolar o primeiro valor da tabela2 que é 5. A consulta será executada com a seguinte lógica: "retorne o valor 5, desde que exista algum retorno no EXISTS, ou seja, desde que seja retornado algum valor de campo da tabela1". A subconsulta retornará TRUE, então o valor 5 será retornado. O mesmo acontecerá para todos os demais elementos de tabela2. Logo, o resultado será uma tabela com os mesmos valores da tabela2.

campo
5
5
7
8
8
9

A situação seria diferente caso houvesse alguma condição de comparação:

select campo from tabela2

where exists

(select campo from tabela1 where tabela1.campo=tabela2.campo)

37- (CESPE / CEBRASPE - 2022 - TCE-SC - Auditor Fiscal de Controle Externo).

tabela1
campo
0
2
3
3
4
4
6
7

tabela2
campo
5
5
7
8
8
9

Considerando as tabela1 e tabela2 apresentadas, julgue o item que se segue, referentes a banco de dados.

Considere que o comando a seguir seja executado sem erro.

select campo from tabela2

where exists

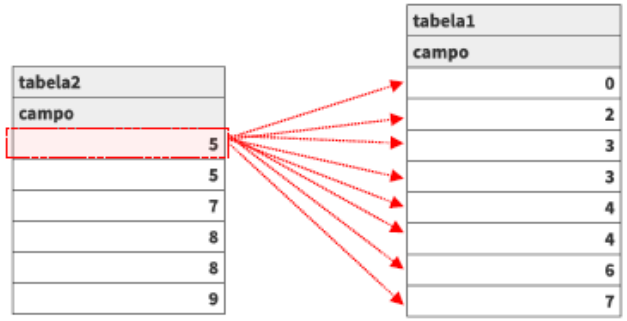
(select campo from tabela1)

Nesse caso, o resultado será a tabela seguinte.

campo
5
5
8
8
9

Vejamos como seria a execução:

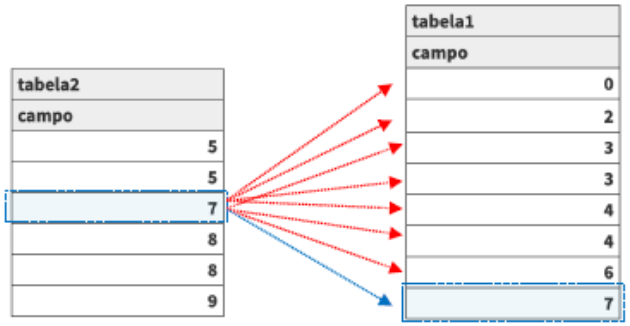
1. Isolando o primeiro valor da tabela2 e comparando-o com todos os resultados da subconsulta (que são simplesmente os valores da tabela1).



Como não existe nenhum valor correspondente, então o EXISTS retorna FALSE, e o SELECT não será executado para essa primeira linha de valor = 5.

2. Da mesma forma, já sabemos que não será executado também para a segunda linha.

3. Agora vejamos o que acontece na terceira linha:



Nesse caso, o valor 7 existe em algum dos valores da subconsulta, então o EXISTS retorna TRUE e o SELECT será executado para a linha de valor 7.

4. O processo seguirá para todos os registros da tabela2, mas nós já podemos perceber que os valores que serão retornados são aqueles da tabela2 que também existem na tabela1. Portanto somente o 7.

Verdadeiro ou falso?

Contudo, como vimos, o comando da questão retorna 5, 5, 7, 8, 8 e 9.

Gabarito: Errado.

37- (CESPE / CEBRASPE - 2022 - TCE-SC - Auditor Fiscal de Controle Externo).

tabela1
campo
0
2
3
3
4
4
6
7

tabela2
campo
5
5
7
8
8
9

Considerando as tabela1 e tabela2 apresentadas, julgue o item que se segue, referentes a banco de dados.

Considere que o comando a seguir seja executado sem erro.

select campo from tabela2

where exists

(select campo from tabela1)

Nesse caso, o resultado será a tabela seguinte.

campo
5
5
8
8
9

O resultado da assertiva seria obtido com a seguinte comparação:

select campo from tabela2

where **not** exists

(select campo from tabela1 where tabela1.campo=tabela2.campo)

Verdadeiro ou falso?

Qual a diferença entre IN e EXISTS?

Dada a tabela Produtos a seguir:

ProdutoID	Nome_do_Produto	FornecedorID	CategoriaID	Unidade	Preco
1	Leite	1	1	Litros	3
2	Banana	1	1	Kilogramas	5
3	Melancia	1	2	Unidade	6
4	Pão	2	2	Pacote	4
5	Suco	2	2	Litros	8

E a tabela Produtos Líquidos a seguir:

ProdutoID	Nome_do_Produto
1	Leite
5	Suco

Liste os nomes dos produtos que são líquidos

1

```
SELECT p.Nome_do_Produto
FROM Produto p
WHERE p.ProdutoID IN (SELECT Produtos_Liquidos.ProdutoID FROM Produtos_Liquidos)
```

2

```
SELECT p.Nome_do_Produto
FROM Produto p
WHERE EXISTS (SELECT * FROM Produtos_Liquidos pl WHERE pl.ProdutoID = p.ProdutoID)
```